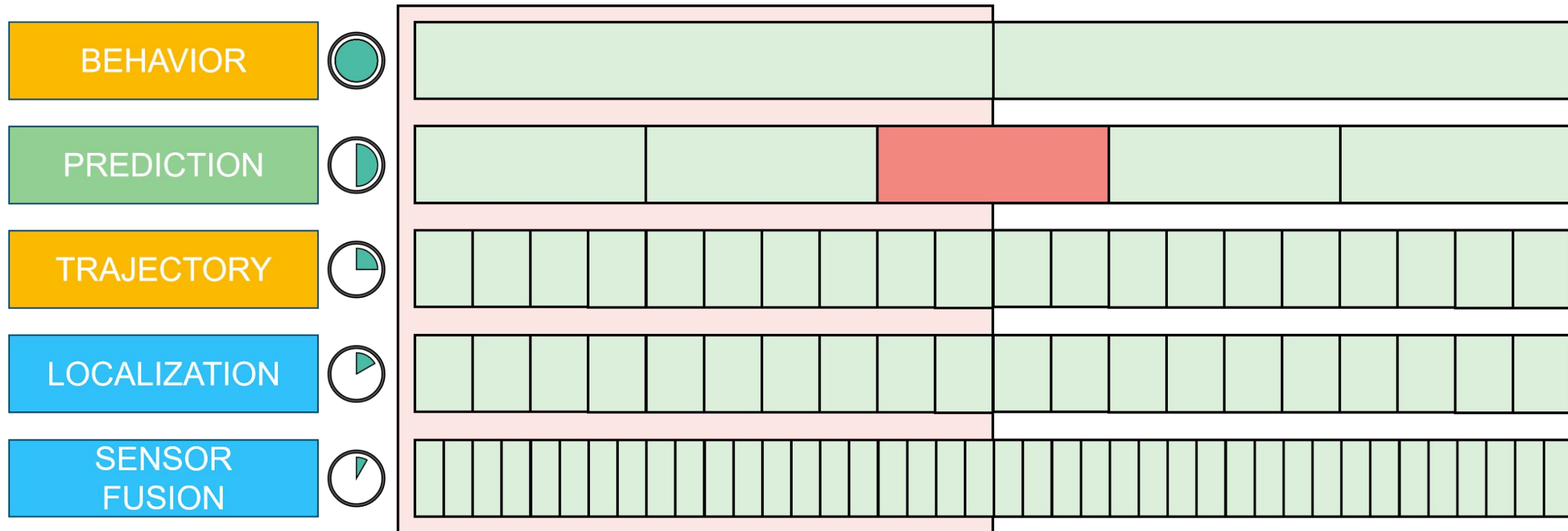


Scheduling Compute Time



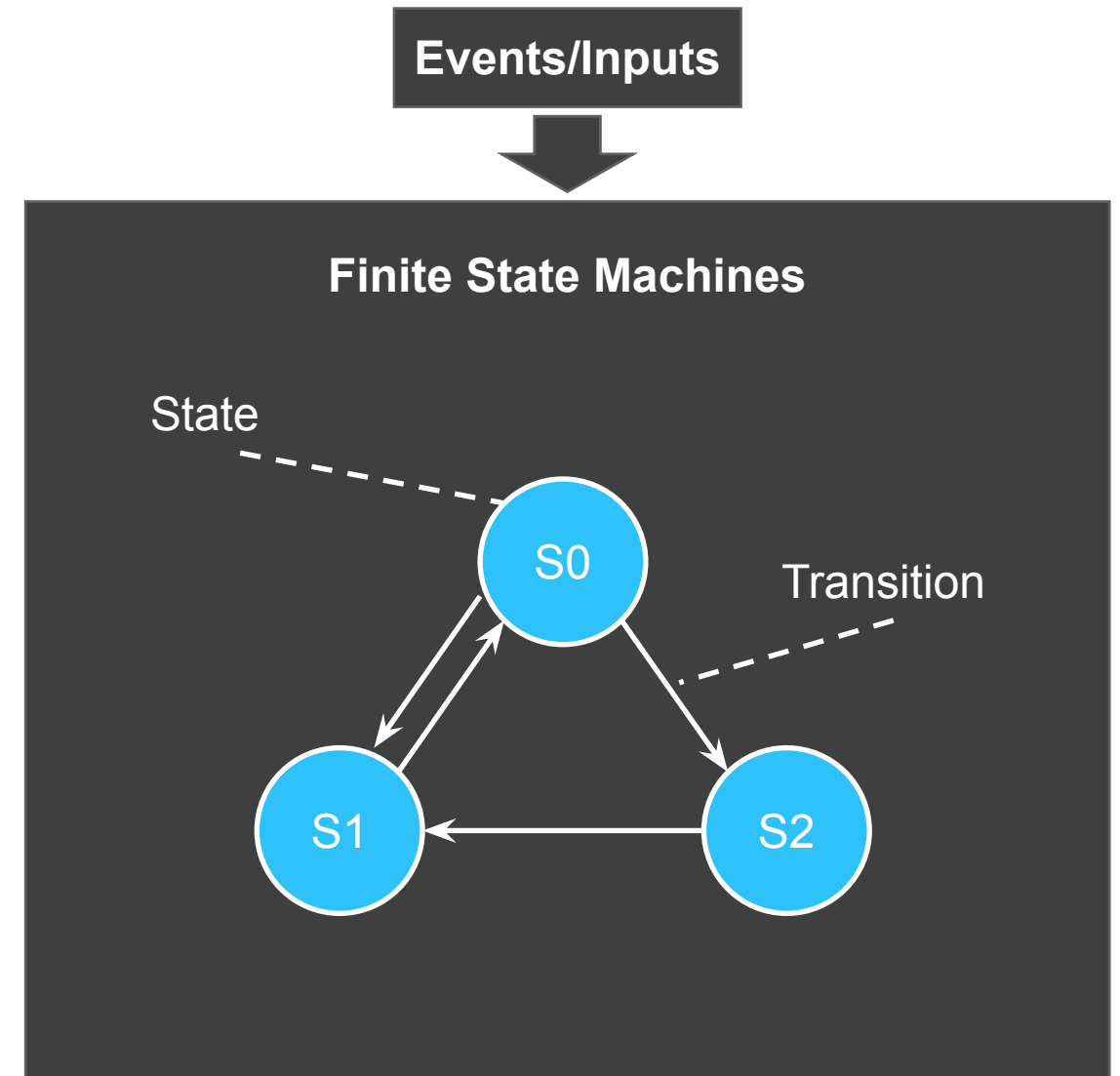
Finite State Machines (FSM)

Finite State Machines (FSMs) are a type of rule-based system that makes decisions based on a finite number of states.

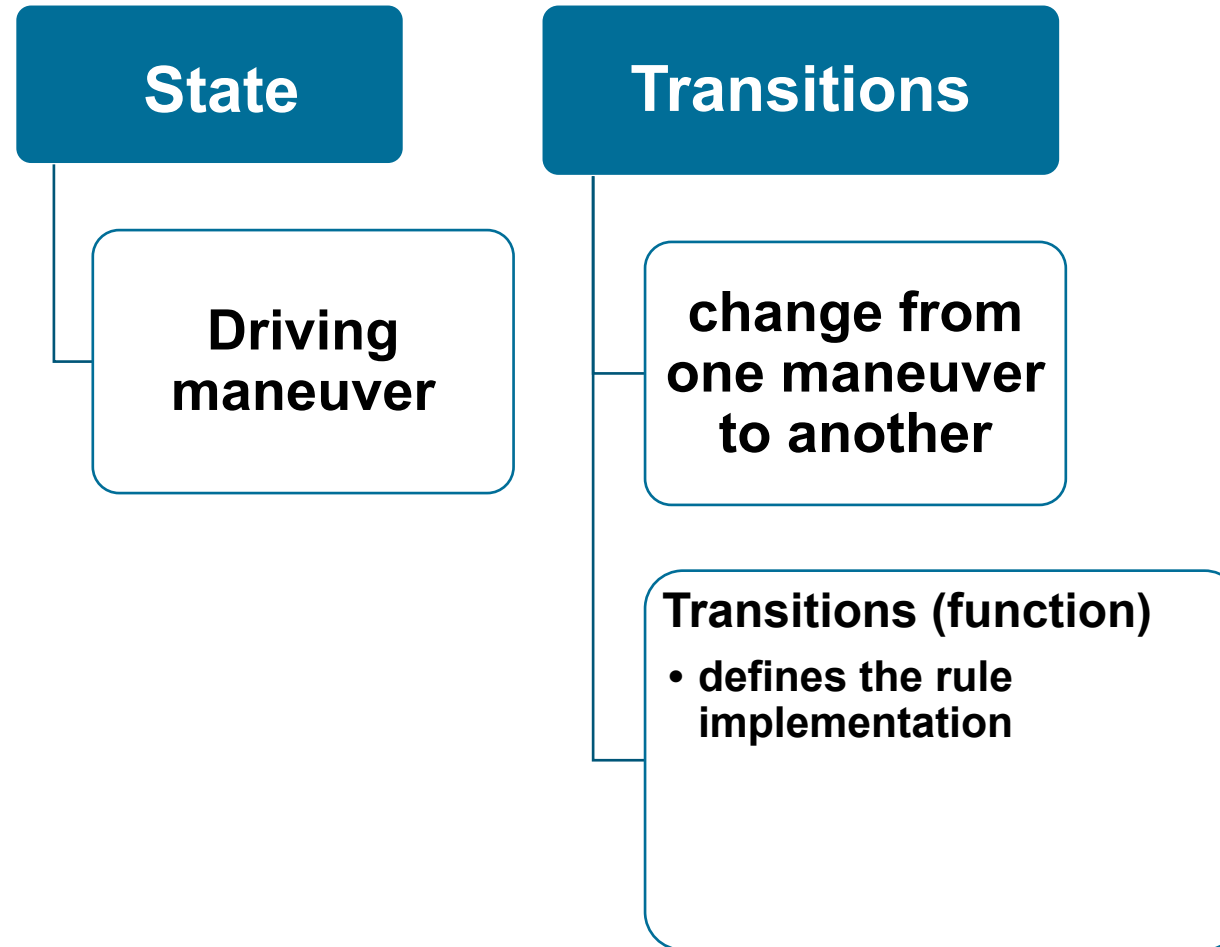
State: condition or mode in which the system can be (e.g., keep lane)

Event: something that triggers a transition from one state to another. In our case, an input information

Transition: A transition is a change of state that occurs when an event is detected (e.g., from lane keeping to merging)



Finite State Machines (FSM) in Autonomous Vehicles



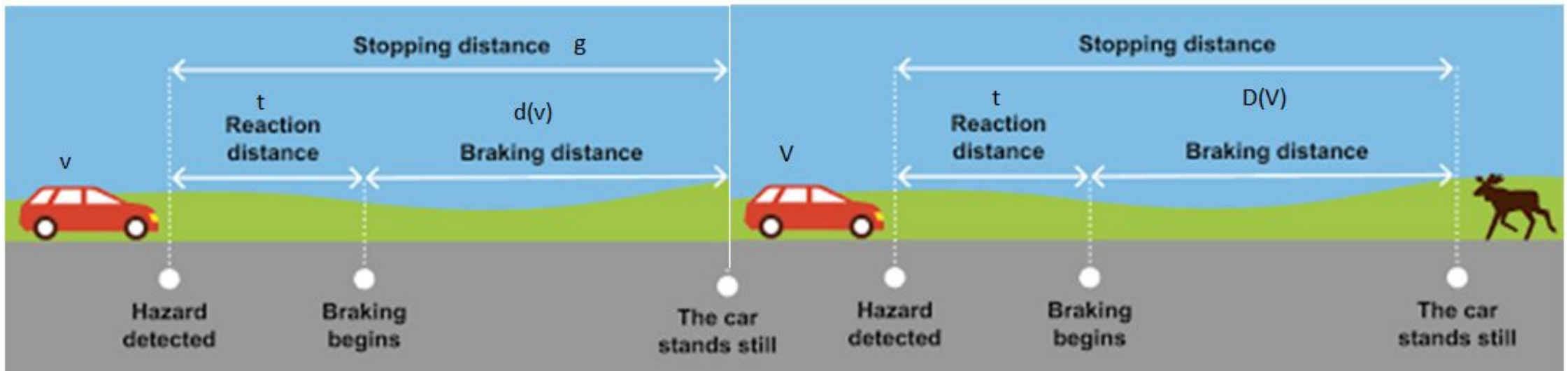
Car Following Model

- Own speed: v
- Speed difference to lead car: $\Delta v = V - v$
- Distance to lead car: g

Vehicle is safe, if the braking distances $d(v)$ (own) and $D(v)$ (lead) fulfil:

$$dv + v\tau \leq DV + g$$

Where τ is the minimum safe headway, driver are willing to accept (may be related to their reaction time) ¹



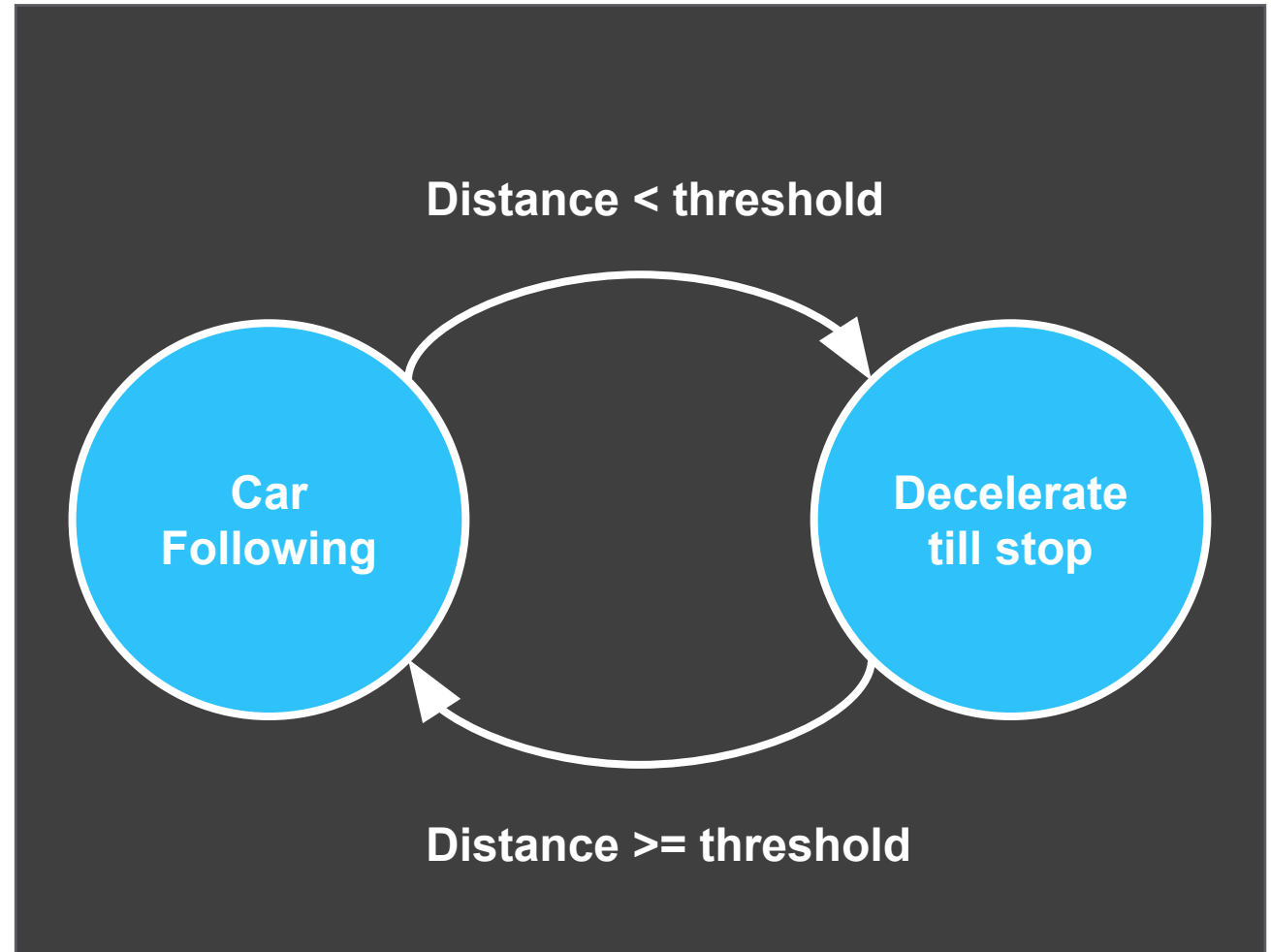
Adapted from <https://korkortonline.se/en/theory/reaction-braking-stopping/>

1] "Microscopic Modelling of Traffic Flow: Investigation of Collision Free Vehicle Dynamics", S. Krauß, DLR (Hauptabteilung Mobilität und Systemtechnik), 1998, ISSN 1434-8454

Finite State Machines (FSM) in Autonomous Vehicles

If a certain distance is lower than a defined threshold, then the following vehicle will decelerate. If the distance is higher or equal the vehicle will continue following the lead vehicle

Example



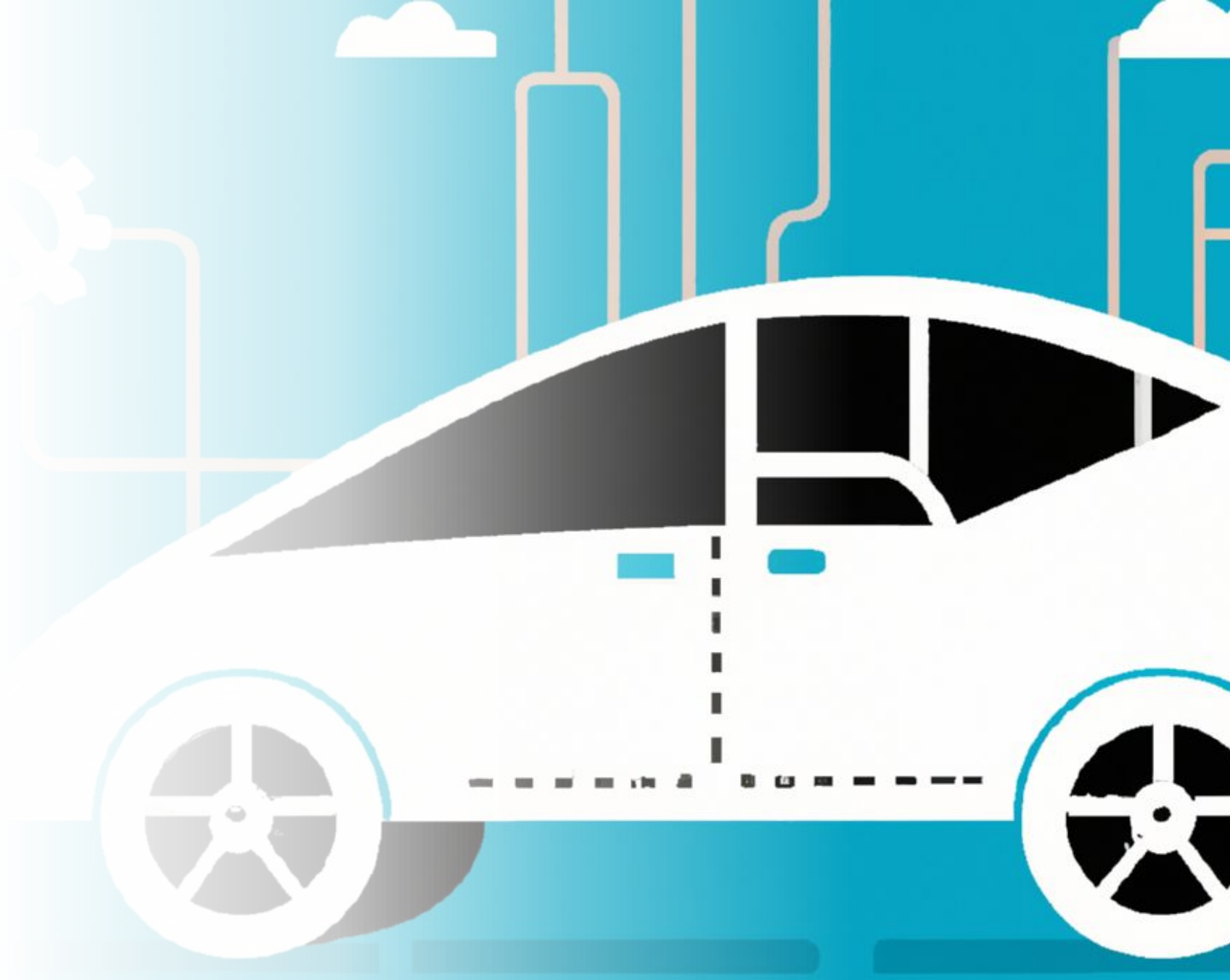
Finite State Machines (FSM)

•Advantages

- Simple and transparent
- Provide formalism and clarity
- Fast and efficient
- Modularity and scalability

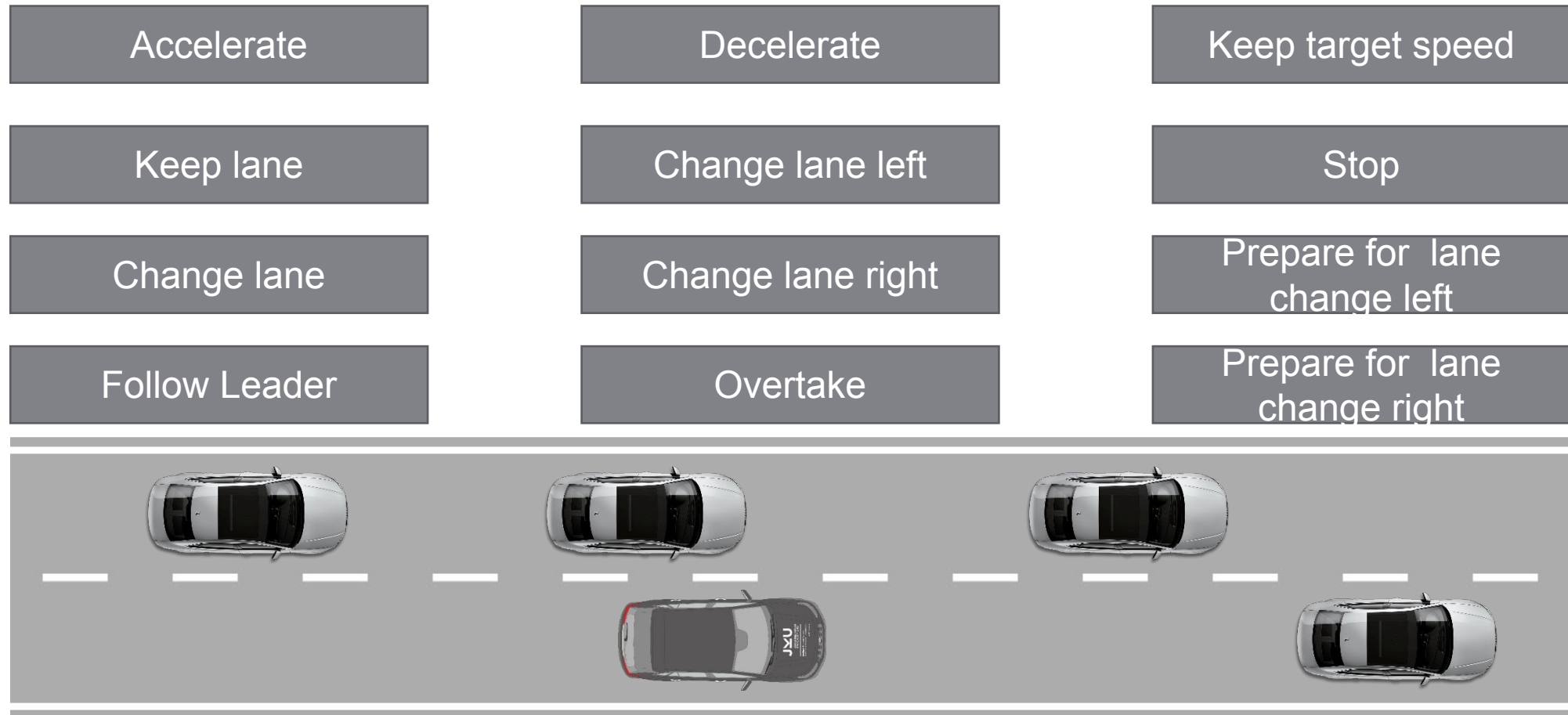
•Disadvantages

- Limited flexibility
- Combinatorial explosion
- Difficult to model complex scenarios
- Limited learning ability



Finite State Machines (FSM)

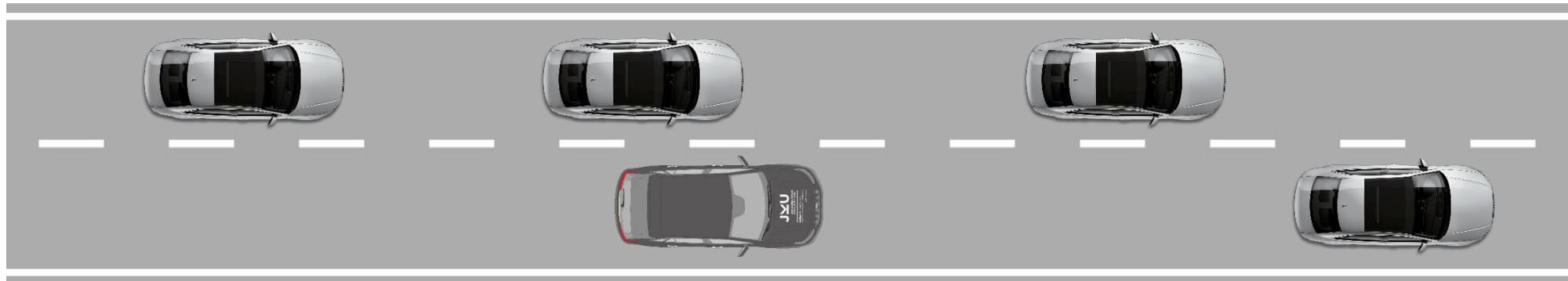
FSM behavior planner for a lane changing maneuver on a highway:



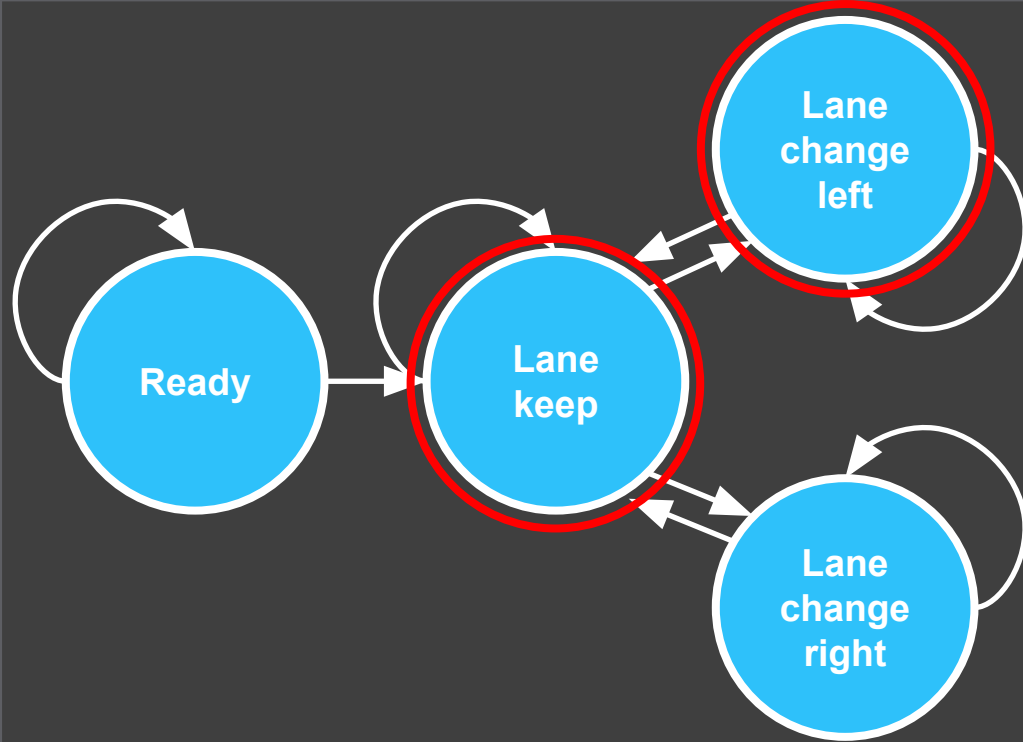
Finite State Machines (FSM)

The states we will use

Accelerate	Decelerate	Keep target speed
Keep lane	Change lane left	Stop
Change lane	Change lane right	Prepare for lane change left
Follow Leader	Overtake	Prepare for lane change right



Finite State Machines (FSM)

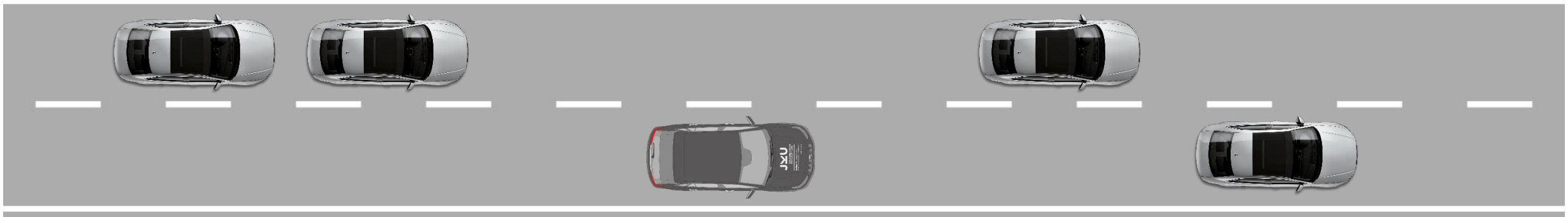


LANE KEEP:

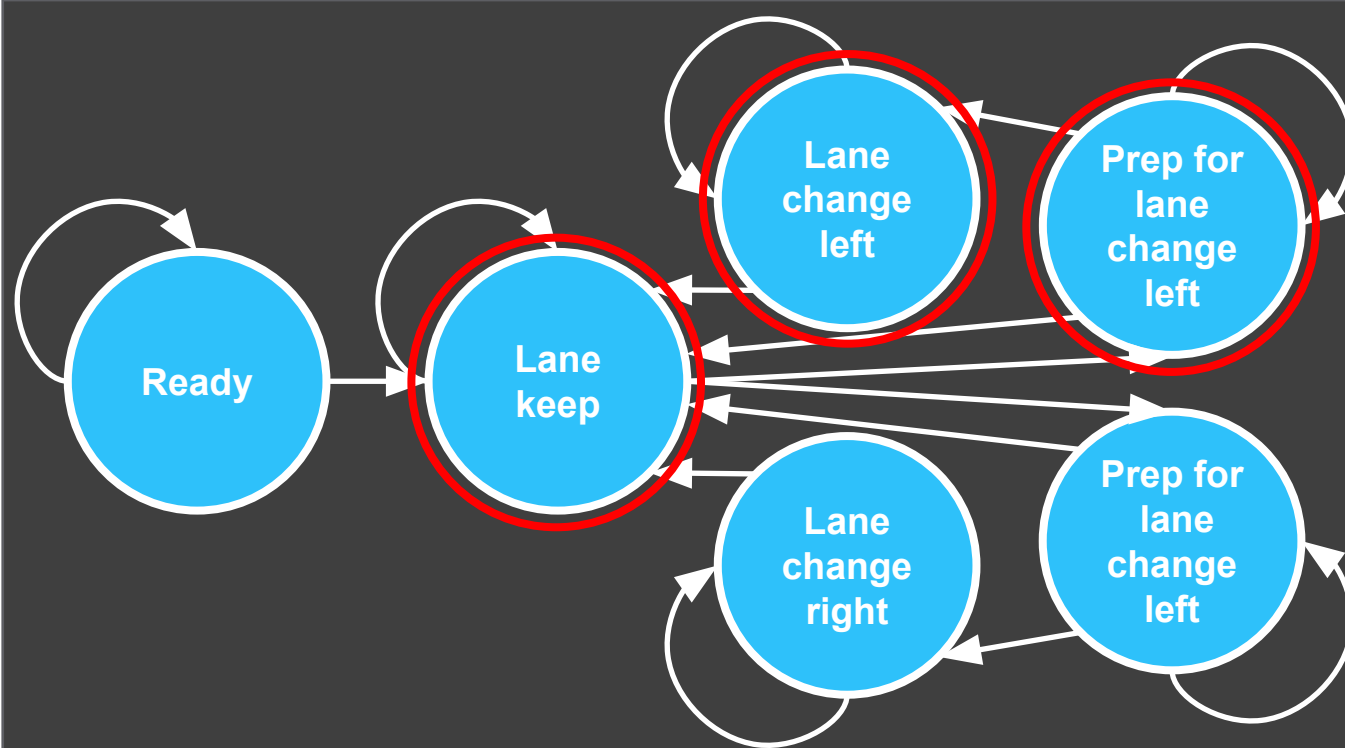
- stay in lane
- drive at target speed when possible, otherwise...
 - drive at a speed limited by other factors

LANE CHANGE LEFT/RIGHT:

- move left or right
- same rules as lane keep

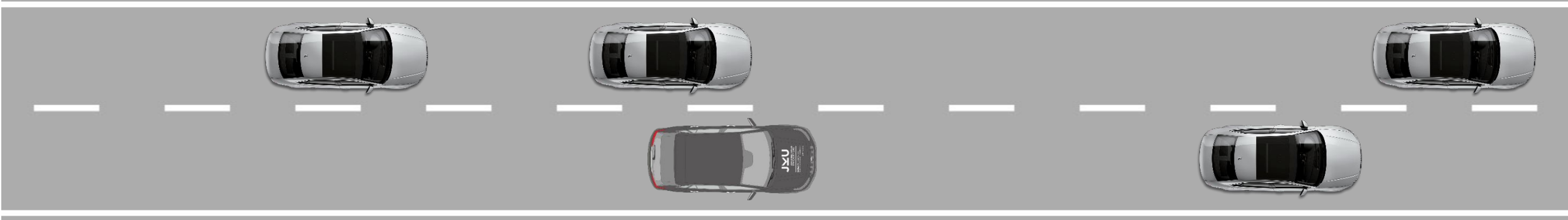


Finite State Machines (FSM)



PREPARE LANE CHANGE LEFT/RIGHT:

- stay in current lane
- attempt to match position and speed of “gap” in lane
- signal - activate turning signal



Finite State Machines (FSM)

Data required for
the transition
functions as
input

- Predictions
- Map
- Speed Limit
- Localization Data
- Current State

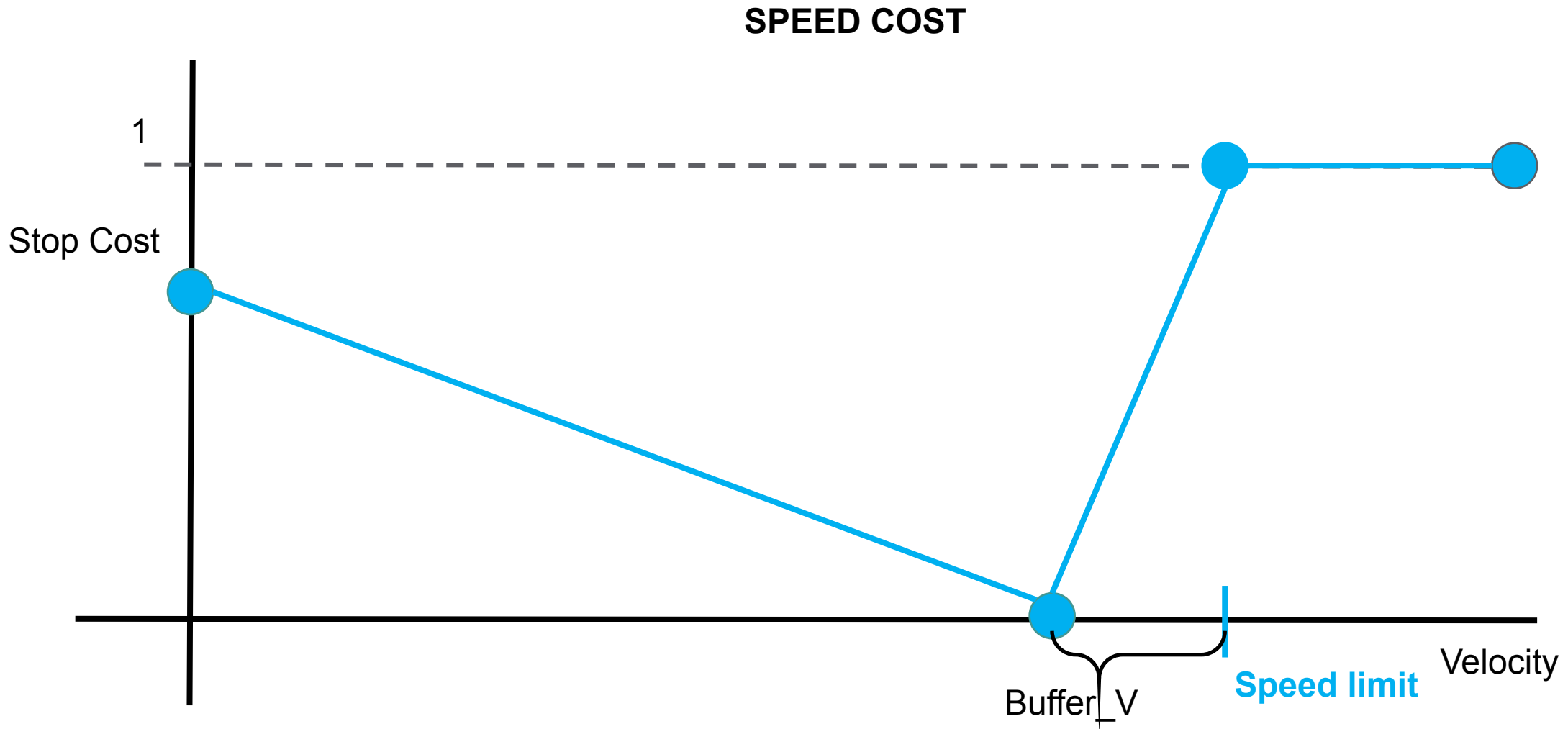
Finite State Machines (FSM): Behavior Planning Pseudocode

Example to implement a transition function

- Generating rough trajectories for each accessible "next state"
- Calculate the costs of each trajectory
- Select the state with the lowest cost trajectory

```
def transition_function(predictions, current_fsm_state, current_pose, cost_functions, weights):  
    # only consider states which can be reached from current FSM state.  
    possible_successor_states = successor_states(current_fsm_state)  
  
    # keep track of the total cost of each state.  
    costs = []  
    for state in possible_successor_states:  
        # generate a rough idea of what trajectory we would  
        # follow IF we chose this state.  
        trajectory_for_state = generate_trajectory(state, current_pose, predictions)  
  
        # calculate the "cost" associated with that trajectory.  
        cost_for_state = 0  
        for i in range(len(cost_functions)) :  
            # apply each cost function to the generated trajectory  
            cost_function = cost_functions[i]  
            cost_for_cost_function = cost_function(trajectory_for_state, predictions)  
  
            # multiply the cost by the associated weight  
            weight = weights[i]  
            cost_for_state += weight * cost_for_cost_function  
        costs.append({'state' : state, 'cost' : cost_for_state})  
  
    # Find the minimum cost state.  
    best_next_state = None  
    min_cost = 9999999  
    for i in range(len(possible_successor_states)):  
        state = possible_successor_states[i]  
        cost = costs[i]  
        if cost < min_cost:  
            min_cost = cost  
            best_next_state = state  
  
    return best_next_state
```

Cost Functions - Example Speed Penalty



Cost Functions - Example Lane Change Penalty

In the image below, the ego car (JKU vehicle) is heading to the target (gold star). It's currently in the correct lane but the car in front is going very slowly, so it considers whether it should perform a lane change (LC) or just keep the lane (KL).

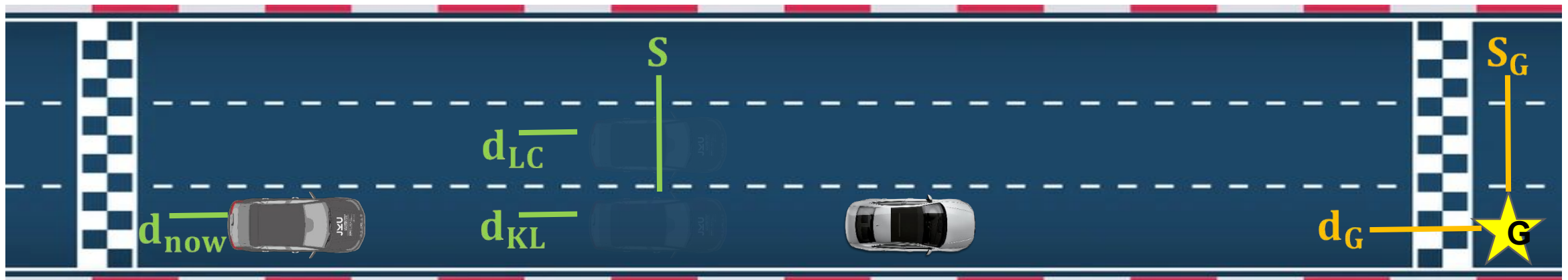
To implement a function that considers the lane choice, it is necessary to first establish what are the relevant variables:

$\Delta s = s_G - s$ the longitudinal distance to arrive to the goal point S_G .

$\Delta d = d_G - d_{LC/KL}$ the lateral distance between the goal lane and the options being considered.

For example:

$\Delta d_{KL} = d_G - d_{KL}$ would be zero and $\Delta d_{LC} = d_G - d_{LC}$ would not.



Cost Functions - Lane Change Penalty

$\Delta s = s_G - s$ the longitudinal distance to arrive to the goal point S_G .

$\Delta d = d_G - d_{LC/KL}$ the lateral distance between the goal lane and the options being considered.

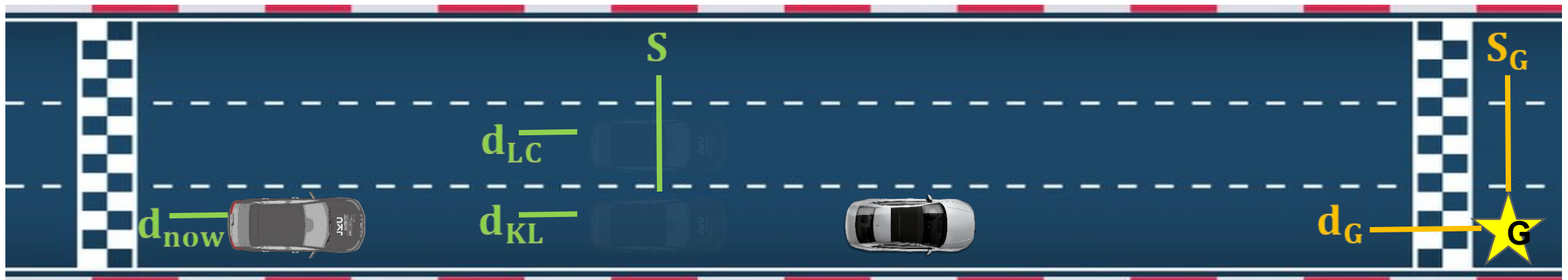
For delta d (Δd): we prefer the absolute value of Δd to be small.

s affects our considerations of lane cost: the costs associated with lane change are more important when we are close to the goal.

So, we need a cost function that penalizes high $|\Delta d|$ and low Δs .

Furthermore, we want to make sure that the **maximum** cost of this cost function never exceeds one and that the **minimum** never goes below zero.

$$\text{cost} = 1 - e^{-\frac{|\Delta d|}{\Delta s}}$$



Cost Function Design: Challenges

Solving new problems without “unsolving” old ones.

- Regression Testing

Balancing costs

Feasibility, Safety, Legality, Comfort, Efficiency

- Weights can change depending on situation

Individual cost functions

- Specification of cost function responsibility
- Types
 - Binary=choosing between 2 options
 - Discrete=between different options
 - Continuous=optimizing a continuous variable such as speed

All the output of the cost function design between -1 and 1

- Parametrization
 - depends on the specific problem being solved and the type of model being used.
- Vehicle state (pos, vel, acc)

Cost Functions: Binary, discrete and continuous

High cost for imminent collision (i.e., the predicted output is 1), and if low cost for no collision (i.e., the predicted output is 0).

Inputs:

- distance between the car and the obstacle,
- the speed of the car,
- the time to collision.

number of cars in each lane,

speed of cars in the vicinity,

distance to the destination.

Lowest cost (i.e., the predicted output is the optimal speed) when the car is traveling at a speed that minimizes energy consumption while maintaining a safe distance from other cars.

Inputs:

- road gradient,
- wind speed and direction,
- traffic density,
- road surface conditions.

Cost Functions: Examples for safety, efficiency and comfort

Safety Cost Function

cost increases as the vehicle approaches a stationary object, and the cost function outputs a maximum value when a collision is imminent

to measure the level of risk associated with a particular driving behavior or action.

Parameters:

- distance to the closest obstacle,
- the speed of the vehicle,
- the braking distance,
- the time to collision.

Efficiency cost function

cost is minimized when the vehicle is traveling at a speed that maximizes fuel efficiency

to measure the fuel consumption or energy efficiency.

Parameters:

- speed of the vehicle,
- road gradient
- traffic density
- engine load.

Comfort Cost Function

cost is minimized when the vehicle's acceleration and deceleration rates are within a comfortable range for passengers

to measure the ride quality or passenger comfort of a driverless car

Parameters:

- acceleration/deceleration rates
- jerk,
- road surface conditions
- passenger preferences.

Cost Function Cases

Penalizes trajectories that attempt to accelerate at a rate which is not possible for the vehicle.

$$\begin{cases} 1 & \ddot{s} \geq a_{\max} \\ 0 & \ddot{s} < a_{\max} \end{cases}$$

d lateral distance
 $\ddot{s}$ acceleration

Penalizes trajectories that drive off the road.

$$\begin{cases} 1 & d \geq d_{\max} \\ 1 & d \leq d_{\min} \\ 0 & d_{\min} < d < d_{\max} \end{cases}$$

Cost Function Cases

Penalizes trajectories that exceed the speed limit.

$$\begin{cases} 1 & \dot{s} \geq v_{\text{speed limit}} \\ 0 & \dot{s} < v_{\text{speed limit}} \end{cases}$$

Penalizes trajectories that do not stay near the center of the lane.

$$\frac{1}{1 + e^{-(d - d_{\text{lane center}})^2}}$$

Rewards trajectories that stay near the target lane.

$$(\text{lane number} - \text{target lane number})^2$$

Implement a Behavior Planner

In this exercise (see **EX_Planning.zip part I (In Moodle)**) you will implement a **behavior planner** and **cost functions** for driving on a highway scenario. The planner will **use prediction data** to set the state of the ego vehicle to one of 5 values and generate a corresponding vehicle trajectory:

- **KL** - Keep Lane
- **LCL/LCR** - Lane Change Left / Lane Change Right
- **PLCL/PLCR** - Prepare Lane Change Left / Prepare Lane Change Right

The objective of this task is to navigate through traffic and arrive to the goal in the shortest time as possible. Note that the goal lane and s value, as well as the traffic speeds for each lane, are set in ***simulate_behavior.py***.

Since the goal is in the slowest lane, in order to get the lowest time, you'll want to choose cost functions and weights to drive in faster lanes when appropriate. We've provided two suggested cost functions in ***cost_functions.py***.

Implement a Behavior Planner

Instructions:

- Implement the ***choose_next_state*** function in ***vehicle.py***. You can use the Behavior Planning Pseudocode page in the current lesson as a guideline for your implementation.
- In this task, there are a couple of small differences from that pseudocode: you'll be returning a best trajectory corresponding to the best state instead of the state itself. Additionally, the function inputs will be slightly different in this quiz than in the classroom concept.
- For this part, we have provided several useful functions:
 1. ***successor_states()*** - Uses the current state to return a vector of possible successor states for the finite state machine.
 2. ***generate_trajectory()*** - Returns a vector of Vehicle objects representing a vehicle trajectory, given a state and predictions. Note that trajectory vectors might have size 0 if no possible trajectory exists for the state.
 3. ***calculate_cost()*** - Included from *cost_functions.py*, computes the cost for a trajectory.

Implement a Behavior Planner

- Choose appropriate weights for the cost functions in ***cost_functions.py*** to induce the desired vehicle behavior. Two suggested cost functions have been implemented but you are free to experiment with your own cost functions. The ***get_helper_data()*** function in ***cost_functions.py*** provides some preprocessing of vehicle data that may be useful in your cost functions. See if you can get the vehicle to move into a fast lane for a portion of the track and then move back to reach the goal!
- Run ***python simulate_behavior.py*** and see how your car does! How fast can you get to the goal?